List of dunder methods

Mostly, but not entirely Complete
Dr. J, Fall 2026

There are other dunder names (such as `__name__`) that are not methods, those are ***not*** on this list.

I am kind of conflicted about what order to put them in, logical or alphabetical. I'm going to go with alphabetical, and try to have a 'See also' section for the ones that are logically connected.

For more information on any of these, search the Python Documentation, and look in the results for a link that looks like `object.__method__`. This is not always the first link, but is most often the one you should be looking for.

Some methods have to return something of a specific type, I've indicated those with type annotations. In many cases, you can create overloaded behavior that describes how to add things of different data types (as is done with, for ex `3 + 5.0`, that is because of a function overload in the `int` data type that defines how to add an integer to a float).

I will use the variable name `object` to represent an instance of a class in places where the code would appear outside the class definition.

`__abs__(self)`
- Implements what `abs(object)` does. Intended for absolute value.

`__add__(self, other)`
- Implements what `self + other` does. Intended for addition.
- See also: `__radd__`, `__iadd__`.

`__alloc__(self) -> int`
- Used to indicate size of the *contents* of an object, as far as I can tell. Differs from `__sizeof__` because `__sizeof__` includes the size of the base object as well as the contents. I'm guessing this is essentially obsolete, replaced by `__sizeof__`.
- This method literally turns up zero results in the Python documentation. It is implemented in the `bytearray` type, that's the only reason I know of its existence. It only appears in 3 locations in the Python source code, and two of them are unit tests for the first, so `bytearray` is the *only* place it appears in all of Python.
- See also: `__sizeof__`.

`__and__(self, other)`
- Implements what `self & other` does. This is intended to be the bitwise AND operation on
- binary numbers `self` and `other`.

- See also `__invert__`, `__or__`, `__xor__`, `__lshift__`, `__rshift__`, and various `__i[operation]__` methods including `__iand__`.

`__await__()`

- Used with the `async` package, I'm going to skip the details.

`__bool__(self) -> bool`

- Used to implement the type conversion function `bool(object)`.
- Also used if you were to do something like `if object:` to determine when the object evaluates as `True` or `False`. Typically, you should make the object evaluate to `False` if it is the zero object, or trivial somehow. So, for example, the empty list `[]` evaluates to `False`, but a list that has any members evaluates to `True`.
- If this method would only return `True` in the circumstance where `__len__(self)` is defined and returns `0`, then this should not be defined because that will happen automatically.

`__buffer__(self, flags) -> memoryview`

- Used to give efficient access to underlying data, most notably in `bytes` objects.
- Must return a `memoryview` object.
- See also: `__release_buffer__`.

`__bytes__(self) -> bytes`

- Used to implement the type conversion function `bytes(object)`.
- The function `pickle.dumps()` can make a useful conversion, but - that serialization isn't always what you want – there might be a more compact way to represent the object. For example, if `x` represents an integer, then `pickle.dumps(x)` creates a binary representation of the Python object that `x` represents, but you'd probably really just want to return the number converted from base 10 to binary.

`__call__(self)`

- Used to implement what to do if an instance is called like a function.
- For example, if you have `class A:...` and you create an instance `a = A()`, then, if `A` provides a `__call__` method, then `a` is a function, so you can call it like: `b = a()`.
- Any arguments on `__call__` after `self` are interpreted as arguments that apply to every instance. -

`__ceil__(self) -> int`

- Implements what `ceil(object)` and what `math.ceil(object)` does. Intended to round up to the first greater integer.
- Could return an `Integral` object if you are using number types that are superclasses of `Integral` such as `Real` or `Rational`.
- See also `__floor__`.

`__complex__(self) -> complex`

- Used to implement the type conversion function `complex(object)`. Intended to create complex numbers (those with real and imaginary parts). Notice that Python uses `j` for `sqrt(-1)`.

`__contains__(self, item)`

- Used to implement `item in object`.

`__copy__(self)`

- Used to implement `copy.copy()`.
- This a ***shallow*** copy, see this page for more on shallow vs. deep copies. The TL,DR on constructing a shallow copy is that you should instruct Python to create a new instance, but pointers to any objects inside the copy. Deep copies should create a new version of the original and also all objects inside the original.
- See also: `__deepcopy__`.

`__del__(self)`

- Used when you call `del object`. You should probably just not mess with this at this point. Whatever is here will happen before the object actually gets deleted. This may or may not get called as the program exits (i.e. during garbage collection). If you are using class inheritance, there is some more stuff to think about. You can also 'resurrect' the object with this method, but you probably shouldn't, I'm not going to get into the details here.
- See also: `__delattr__`.

`__delattr__(self, name)`

- Used when you call `del object.name`. Normally, a program is allowed to remove one or more attributes from an object; this would be useful in preventing that if you wanted to make sure an attribute can't be deleted without deleting the entire object.
- See also: `__del__`, `__getattr__`, `__getattribute__`, `__setattr__`.

`__deepcopy__(self)`

- Used to implement `copy.deepcopy()`. See `__copy__` for details.

`__delitem__(self, key)`

- Implements what `del object[key]` does.
- The three methods `__delitem__`, `__getitem__`, and `__setitem__` make your object into a sequence object (like a list or a tuple).
- See also: `__getitem__`, `__setitem__`.

`__dir__(self)`

- Implements what `dir(object)` does. This object must return some kind of sequence (e.g. a `list`, `string` or `tuple`). Typically used to return a list of all attributes and methods, an abbreviated version of what the builtin `help()` function does.

`__divmod__(self, other)`

- Implements what `divmod()` does. `divmod()` is intended for breaking an integer into two parts, `self // other` (quotient) and `self % other` (remainder), both are returned as a 2-tuple. For example, `divmod(13, 5)` returns `(2, 3)` because 5 goes into 13 twice with remainder 3.
- See also: `__floordiv__`, `__mod__`.

`__enter__(self)`

- Used with the `with` keyword to establish a context manager. The return value of `__enter__` is assigned to the variable after the keyword `as`. So, in: `with open(filename, 'r') as f:` the `open()` function returns an object that interfaces with an open text file; that class is called `_io.TextIOWrapper`. Then

`_io.TextIOWrapper.__enter__()` just returns `self` so that the instance gets assigned to `f`.
- See also: `__exit__`.

`__eq__(self, other) -> bool`
- Implements what `self == other` does.
- Only implemented if `self == other` should be different than `self is other`.
- You can return something other than a boolean, but under most circumstances, Python will call `bool()` on the return value anyway.
- See also: `__ge__`, `__gt__`, `__le__`, `__lt__`, and a variety of `__i[operation]__` methods.

`__exit__(self, exc_type, exc_value, traceback)`
- Used at the end of a `with` block. Runs code that closes out the context manager.
- So, in: `with open(filename, 'r') as f:` the `__exit__()` method runs `f.close()`.
- See also: `__enter__`

`__float__(self) -> float`
- Used to implement the type conversion function `float()`.

`__floor__(self) -> int`
- Implements what `floor(object)` and what `math.floor(object)` does. Intended to round down to the first smaller integer.
- Could return an `Integral` object if you are using number types that are superclasses of `Integral` such as `Real` or `Rational`.

`__floordiv__(self, other)`
- Implements what `self // other` does.
- See also: `__divmod__`, `__mod__`.

`__format__(self, format_spec) -> str`
- Used with formatted strings, using any of the different string formatting approaches.
- The second argument should use the String Format Mini-Language.
- It is expected that `object.__format__(x, '')` should be equivalent to `__str__()`.
- See also: `__str__`, `__repr__`.

`__ge__(self, other)`
- Implements what `self >= other` does.
- Notice that, if you want all of the comparison operators to work, this ***does*** need to be implemented separately, even if `__gt__` and `__eq__` are both implemented. Python does ***not*** default this to `__gt__ or __eq__`.
- You can use `@functools.totalordering` as a shortcut to implementing all the comparisons, though it warns that it is not computationally efficient.
- You can return something other than a boolean, but under most circumstances, Python will call `bool()` on the return value anyway.
- See also: `__eq__`, `__ge__`, `__le__`, `__lt__`.

`__getattr__(self, name)`

- Used for access to attribute values that are not directly accessible. There are several technicalities with this, you should do some research and experimentation to make sense of this should you choose to use it. You should probably just use `__getattribute__`.
- See also: `__getattribute__`, `__setattr__`, `__delattr__`.

`__getattribute__(self, name)`

- Standard method for accessing attribute values. There are several technicalities with this, you should do some research and experimentation to make sense of this should you choose to use it.
- See also: `__getattr__`, `__setattr__`, `__delattr__`.

`__getitem__(self, index)`

- Used to handle `object[index]`.
- The `index` should be an integer or a slice object if your class is a sequence, or something that could be a dictionary key if your object is a collection. Support for slices is optional.
- The three methods `__delitem__`, `__getitem__`, and `__setitem__` make your object into a sequence object (like a list or a tuple) or a collection (like a dictionary).
- See also: `__setitem__`, `__delitem__`.

`__getnewargs__()` and `__getnewargs_ex__()`

- Used in unpickling, provides arguments to be passed to `__new__` as the object is created. The `__getnewargs_ex__` method is only used for backwards compatibility.

`__getstate__(self)`

- Used in pickling, can give you more control in how your object is serialized.

`__gt__(self, other)`

- Implements what `self > other` does.
- You can return something other than a boolean, but under most circumstances, Python will call `bool()` on the return value anyway.
- See also: `__eq__`, `__ge__`, `__le__`, `__lt__`.

`__hash__(self) -> int`

- Implements what `hash(object)` does. Requires that `__eq__` be defined. If `object1 == object2` is `True`, then `__hash__` must be defined so that `hash(object1)` is the same as `hash(object2)`.

`__iadd__(self, other)`

- Implements what `self += other` does. Reassigns `self` with the return value instead of actually returning something.
- Should only be implemented if this needs to be different from `self = self + other`.
- See also: `__add__`, various other `__i[operation]__` methods.

`__iand__(self, other)`

- Implements what `self &= other` does. Reassigns `self` with the return value instead of actually returning something.
- Should only be implemented if this needs to be different from `self = self & other`.

- See also: `__and__`, `__invert__`, `__or__`, `__xor__`, `__lshift__`, `__rshift__`, and various other `__i[operation]__` methods.

`__ifloordiv__(self, other)`

- Implements what `self //= other` does. Reassigns `self` with the return value instead of actually returning something.
- Should only be implemented if this needs to be different from `self = self // other`.
- See also: `__floordiv__`, various other `__i[operation]__` methods.

`__ilshift__(self, other)`

- Implements what `self <<= other` does. Reassigns `self` with the return value instead of actually returning something.
- Should only be implemented if this needs to be different from `self = self << other`.
- See also: `__and__`, `__invert__`, `__or__`, `__xor__`, `__lshift__`, `__rshift__`, and various other `__i[operation]__` methods.

`__imatmul__(self, other)`

- Implements what `self @= other` does. Reassigns `self` with the return value instead of actually returning something.
- Should only be implemented if this needs to be different from `self = self + other`.
- See also: `__matmul__`, various other `__i[operation]__` methods.

`__imod__(self, other)`

- Implements what `self %= other` does. Reassigns `self` with the return value instead of actually returning something.
- Should only be implemented if this needs to be different from `self = self % other`.
- See also: `__mod__`, various other `__i[operation]__` methods.

`__imul__(self, other)`

- Implements what `self *= other` does. Reassigns `self` with the return value instead of actually returning something.
- Should only be implemented if this needs to be different from `self = self * other`.
- See also: `__mul__`, various other `__i[operation]__` methods.

`__index__(self)`

- This does ***NOT*** implement `object[index]`, if that's what you're looking for, use `__getitem__`.
- Used to convert an object into a positive integer, used as a fallback if `__int__` is not defined.
- Intended for use in places where integers are generally required. So, if you wanted your object to be able to be used as one entry in list slice notation, you'd implement `__index__` to make Python recognize it that way: `my_list[obj1:obj2]` makes sense if `obj1` and `obj2` are instances of a class with the `__index__` method defined.

`__init__(self, *args, **kwargs)`

- You know and love this one. Sets up a new instance of your class. Technically, does not actually create it, that job belongs to `__new__`.

- Generally, all the user-customizable work can and should be done in `__init__`, leaving `__new__` unchanged.

`__init_subclass__(cls)`

- Used to control aspects of how new subclasses inherit properties from a superclass.

`__int__(self) -> int`

- Used to implement the type conversion function `int()`.

`__invert__(self)`

- Implements what `~self` does. This is intended to be the bitwise NOT operation on binary number self.
- See also `__and__`, `__or__`, `__xor__`, `__lshift__`, `__rshift__`.

`__ior__(self, other)`

- Implements what `self |= other` does. Reassigns `self` with the return value instead of actually returning something.
- Should only be implemented if this needs to be different from `self = self | other`.
- See also: `__and__`, `__invert__`, `__or__`, `__xor__`, `__lshift__`, `__rshift__`, and various other `__i[operation]__` methods.

`__ipow(self, other[, modulo])`

- Implements what `self **= other` does. Reassigns `self` with the return value instead of actually returning something.
- Should only be implemented if this needs to be different from `self = self ** other`.
- See also: `__pow__`, and various other `__i[operation]__` methods.

`__irshift__(self, other)`

- Implements what `self >>= other` does. Reassigns `self` with the return value instead of actually returning something.
- Should only be implemented if this needs to be different from `self = self >> other`.
- See also: `__and__`, `__eq__`, `__invert__`, `__or__`, `__xor__`, `__lshift__`, `__rshift__`, and various other `__i[operation]__` methods.

`items(self) -> list | dictview`

- Strictly not a dunder method (along with its cousins, `keys()` and `values()`), but I'll count each as one for the purposes of the assignment. If your class is a dictionary-like collection, then you should implement these three methods. This method should return the list of items in the list; each item in this list is a 2-tuple `(key, value)`. If you're really good, you should not return a `list`, you should return a `dictview` object because that will be linked to the original dictionary so that, if the dictionary changes, then the `dictview` object automatically receives an update and changes to match what happened to the dictionary.
- See also `items(), values()`.

`__isub__(self, other)`

- Implements what `self -= other` does. Reassigns `self` with the return value instead of actually returning something.

- Should only be implemented if this needs to be different from `self = self - other`.
- See also: `__sub__`, `__eq__`, various other `__i[operation]__` methods.

`__iter__(self)`

- For making your class an iterator. The returned object must implement the `__next__` method. See `__next__` for a link and more information.
- See also `__index__`, `__next__`.

`__itruediv__(self, other)`

- Implements what `self /= other` does. Reassigns `self` with the return value instead of actually returning something.
- Should only be implemented if this needs to be different from `self = self / other`.
- See also: `__sub__`, `__eq__`, various other `__i[operation]__` methods.

`__ixor__(self, other)`

- Implements what `self ^= other` does. Reassigns `self` with the return value instead of actually returning something.
- Should only be implemented if this needs to be different from `self = self ^ other`.
- See also: `__and__`, `__eq__`, `__or__`, `__xor__`, `__lshift__`, `__rshift__`, and various other `__i[operation]__` methods.

`keys(self) -> list | dictview`

- See `items()` for more details. This should return a `list` or `dictview` of the keys.
- See also `items()`, `values()`.

`__le__(self, other) -> bool`

- Implements what `self <= other` does.
- Notice that, if you want all of the comparison operators to work, this ***does*** need to be implemented separately, even if `__lt__` and `__eq__` are both implemented, Python does ***not*** default this to `__lt__ or __eq__`.
- See also: `__eq__`, `__ge__`, `__gt__`, `__lt__`.

`__len__(self) -> int`

- Implements what `len()` does.
- Must return a non-negative integer.

`__length_hint__(self)`

- Gives an estimate of the length of the object, as an integer.
- Must return either a positive integer or `NotImplemented`.
- Generally only implemented if `__len__` is not; this method is never required by anything in the Standard Library.
- If `__len__` is implemented, this should always return exactly `len(self)`.
- Intended for use when interfacing with a language such as C/C++ so that memory for an instance or attribute of the object can be pre-allocated. Not guaranteed to be accurate, nor is it guaranteed to be greater than or less than the actual length. I would suggest that you'd be better off estimating high rather than low to avoid re-allocation or error due to overflow.

`__lshift__(self, other)`

- Implements what `self << other` does. Intended for the case where `self` is a binary number, and `other` is an integer, and this operator shifts the bits of `self` to the left by `other` places, filling in with zeros in newly emptied places.
- See also: `__and__`, `__invert__`, `__or__`, `__xor__`, `__rshift__`.

`__lt__(self, other) -> bool`

- Implements what `self < other` does.
- See also: `__eq__`, `__ge__`, `__le__`, `__lt__`.

`__match_args__`

- This is used in the new Structural Pattern Matching framework (`match-case` statements).

`__matmul__(self, other)`

- Implements what `self @ other` does. Intended for matrix multiplication.
- See also: `__mul__`, `__rmatmul__`, `__imatmul__`.

`__mod__(self, other)`

- Implements what `self % other` does. Intended for modulus (remainder in division).
- Notice that the `str` class uses this for an old-school string formatting, using it to substitute variables, has nothing to do with integer modulus.
- See also: `__divmod__`,`__floordiv__`, `__rmod__`, `__imod__`.

`__mro__(cls)`

- This is a class method, it can only be run on a class, not an instance of a class.
- MRO stands for "Method Resolution Order." It shows the inheritance chain that the class is in. The last time in this chain is always `<class 'object'>`.

`__mul__(self, other)`

- Implements what `self * other` does. Intended for multiplication.
- See also: `__matmul__`,`__rmul__`, `__imul__`.

`__ne__(self, other) -> bool`

- Implements what `self != other` does.
- Only needs to be implemented if it does something different from `not(self == other)`.
- See also `__eq__`.

`__neg__(self)`

- Implements what `-self` does. Intended for finding an additive inverse so that `self + (-self) == 0`.

`__new__(cls, *args, **kwargs)`

- This is a class method. Is run to create a new instance of a class, should generally return an instance of `cls`. Assuming it returns an instance of `cls`, `__init__` is called immediately after. Generally only used as part of a subclass/superclass situation where some parts of the creation process should be inherited.
- Most things you'd be tempted to put in this method can be put in `__init__` and should be put there.

`__next__(self)`

- For making your class an iterator, implements the built-in function `next(self)`.

- See this page of the Python documentation for some explanation of how to use it.
- See also `__iter__`.

`__or__(self, other)`

- Implements what `self | other` does. This is intended to be the bitwise OR operation on binary numbers `self` and `other`.
- See also `__invert__`, `__and__`, `__xor__`, `__lshift__`, `__rshift__`, and various `__i[operation]__` methods including `__ior__`.

`__pos__(self)`

- Implements what `+self` does. I'm honestly not sure when you would ever need this.
- Should only be implemented if `+self` is different from `self`.

`__pow__(self, other[, modulo])`

- Notice that the `modulo` argument is optional. Implements what `self ** other` and also what `pow(self, other[, modulo])` does. Intended for raising `self` to the power `other`, while using a data type that is common to `self` and `other` if possible.
- Notice that, if you are actually looking for `self ** other % modulo` then using `pow(self, other, modulo)` is ***much*** faster than using the equation `self ** other % modulus` (because Number Theory).
- This does ***not*** implement the similar function `math.pow(x, y)`, which converts `x` and `y` to `float` and then raises `x` to the `y` power. `math.pow()` will always give a `float` as a result.
- See also: `__ipow__`, `__rpow__`.

`__radd__(self, other)`

- Implements what `other + self` does (reversed addition). Intended for addition when defining addition where your class is on the right.
- For example: If `c` is an instance of `MyClass`, and `x` is an `int`, then you can define `c + x` using `__add__`, but you can't define `x + c` using `__add__` because the left entry in the sum is an `int`, so that is the `self` argument meaning that Python will look in the `int` class for how to carry out this addition. The `int` class does not define how to add an instance of `MyClass` to an `int`, so, you need to use `__radd__` to define how these are added, because `__radd__` method moves `self` to the right entry of the sum.

`__rand__(self, other)`

- Implements what `other & self` does (reversed logical AND).
- See `__radd__` for an example of why reversed operations are needed.
- See also `__and__`.

`__reduce__(self)` `__reduce_ex__(protocol)`

- Used in pickling, you probably shouldn't mess with this. The `__reduce_ex__` method is only for backwards compatibility, so you *really* don't need this.

`__release_buffer__(self, buffer)`

- Release any resources used by a `memoryview` created by a call to `__buffer__`.
- See also: `__buffer__`.

`__repr__(self) -> str`

- Provides a Python-readable string representation of an instance of your class. Used to implement the built-in function `repr()`.
- Intended use: If you run `eval(repr(object))`, this should create a copy of `object`.

`__reversed__(self)`

- Assumes that the object is a collection, should return an iterator that goes through the items in reverse order.
- Only needed if an iterator that returns `object[len(object) - 1], object[len(object) - 2], ..., object[0]` would not work; Python will automatically create this iterator if `__reversed__` is not present.

`__rdivmod__(self, other)`

- Implements what `divmod(other, self)` does (reversed `divmod`).
- See `__radd__` for an example of why reversed operations are needed. See also `__divmod__`.

`__rfloordiv__(self, other)`

- Implements what `other // self` does (reversed floor division).
- See `__radd__` for an example of why reversed operations are needed. See also `__floordiv__`.

`__rlshift__(self, other)`

- Implements what `other << self` does (reversed bitwise left shift).
- See `__radd__` for an example of why reversed operations are needed. See also `__lshift__`.

`__rmatmul__(self, other)`

- Implements what `other @ self` does (reversed matrix multiplication).
- See `__radd__` for an example of why reversed operations are needed. See also `__matmul__`.

`__rmod__(self, other)`

- Implements what `other % self` does (reversed modulo).
- See `__radd__` for an example of why reversed operations are needed. See also `__mod__`.

`__rmul__(self, other)`

- Implements what `other * self` does (reversed multiplication).
- See `__radd__` for an example of why reversed operations are needed. See also `__mul__`.

`__ror__(self, other)`

- Implements what `other | self` does (reversed logical OR).
- See `__radd__` for an example of why reversed operations are needed. See also `__or__`.

`__round__(self[, ndigits])`

- Implements what `round()` does.
- Intended to produce an `int` if `ndigits` is omitted or 0, intended to round off a `float` version of self with `ndigits` to the right of the decimal point.

`__rpow__(self, other[, modulo])`

- Implements what `other ** self` does (reversed power).
- See `__radd__` for an example of why reversed operations are needed. See also `__pow__`.

`__rrshift__(self, other)`

- Implements what `other >> self` does (reversed bitwise right shift).
- See `__radd__` for an example of why reversed operations are needed. See also `__rshift__`.

`__rshift__(self, other)`

- Implements what `self >> other` does.
- Intended for the case where `self` is a binary number, and `other` is an integer, and this operator shifts the bits of `self` to the right by `other` places, discarding digits that drop off the right.
- See also: `__and__`, `__invert__`, `__or__`, `__xor__`, `__lshift__`.

`__rsub__(self, other)`

- Implements what `other - self` does (reversed subtraction).
- See `__radd__` for an example of why reversed operations are needed. See also `__sub__`.

`__rtruediv__(self, other)`

- Implements what `other / self` does (reversed true division).
- See `__radd__` for an example of why reversed operations are needed. See also `__truediv__`.

`__rxor__(self, other)`

- Implements what `other ^ self` does (reversed logical XOR).
- See `__radd__` for an example of why reversed operations are needed. See also `__xor__`.

`__setattr__(self, name, value)`

- Sets `self.name` equal to `value`, (usually) even if `name` was not an attribute of the class which existed before this method was called.
- See also: `__delattr__`, `__getattr__`, `__getattribute__`.

`__setitem__(self, index, value)`

- Implements `self[index] = value`.
- The three methods `__delitem__`, `__getitem__`, and `__setitem__` make your object into a mutable sequence object (like a list).
- See also: `__setitem__`, `__delitem__`.

`__set_name__(self, owner, name)`

- Used to assign values to attributes, mostly for internal use only as far as I can tell.

`__sizeof__(self)`

- Used to implement `sys.getsizeof()`, which should return an integer equal to the number of bytes that the object takes up in memory.

`__str__(self)`

- Used to implement the type-conversion function `str()`.
- Intended to provide a human-readable string representation of an instance of your class.

`__sub__(self, other)`

- Implements what `self - other` does.
- Intended for subtraction. ***Does*** need to be implemented even if `__neg__` and `__add__` are implemented, Python will ***not*** combine them to create a subtraction operation using `self + (-other)`.
- See also: `__neg__`, `__add__`.

`__subclasshook__(subclass)`

- Used to connect a class to an abstract base class. For more on this, check out modules `abc` and `collections.abc`.

`__truediv__(self, other)`

- Implements what `self / other` does.
- Intended for division without rounding, including conversion of `int` inputs to `float` even when `self` is exactly divisible by `other`.
- See also: `__floordiv__`.

`__trunc__(self)`

- Implements what `math.trunc()` does.
- Intended to drop the decimal part of a `float` to produce an `int`. Notice that `math.trunc()` always rounds towards zero, so is equivalent to `floor()` if the input is positive, and is equivalent to `ceil()` if the input is negative.
- The function `int()` is equivalent to `floor()` on a `float`.

`values(self) -> list | dictview`

- See `items()` for more details.
- This should return a `list` or `dictview` of the values.
- See also `items()`, `keys()`.

`__xor__(self, other)`

- Implements what `self ^ other` does.
- This is intended to be the bitwise XOR operation on binary numbers self and other.
- See also `__invert__`, `__and__`, `__or__`, `__lshift__`, `__rshift__`, and various `__i[operation]__` methods including `__ixor__`.